

GAMIFIED LEARNING SYSTEM — FULL ARCHITECTURE

Rewind.

★ 1. SYSTEM OVERVIEW

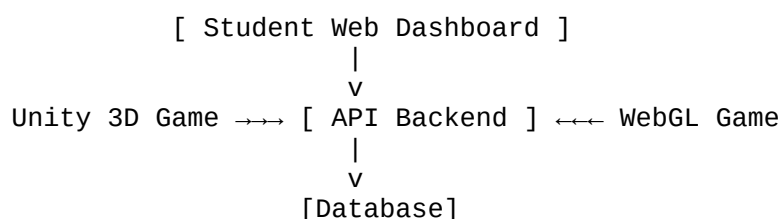
They currently have:

- A Unity 3D game (scenes, gameplay, rewards)
- A WebGL build planned (same game)
- A user dashboard for students
- Separate databases (💀)
- Zero sync between systems (💀💀)
- No central backend (💀💀💀)

You will FIX THIS by building:

- ✓ **ONE backend**
 - ✓ **ONE database**
 - ✓ **One user identity system**
 - ✓ **API endpoints for game + dashboard**
 - ✓ **Score + progress tracking**
 - ✓ **Leaderboard**
 - ✓ **Analytics-ready events**
-

★ 2. PROPOSED SYSTEM DIAGRAM



★ 3. TECH STACK (Recommended)

Backend

- Node.js (Express, NestJS)

Database

- MySQL

Auth

- JWT tokens for game & dashboard
- Role: student, teacher, admin

Unity Integration

Use:

`UnityWebRequest`

to call backend endpoints.

★ 4. DATABASE SCHEMA (Production-Ready)

users

- id (uuid)
- name
- email
- password_hash
- role (student/teacher/admin)
- created_at
- updated_at

game_sessions

- id
- user_id

- start_time
- end_time
- total_score
- level_reached
- rewards_earned
- device_type (unity/webgl)

scores

- id
- user_id
- game_id
- score
- highest_score
- xp
- coins
- last_updated

leaderboards

- id
- user_id
- weekly_score
- monthly_score
- rank

progress

- id
- user_id
- level
- xp
- badges
- completed_lessons (json)

analytics_events

- event_id
- user_id

- event_type
 - event_data (json)
 - timestamp
-

5. API ENDPOINTS (Unity + Web Dashboard)

Auth

POST /auth/register
POST /auth/login
POST /auth/logout

User

GET /user/me
PUT /user/me

Game Session

POST /game/start
POST /game/end

Score

POST /game/score/update
GET /game/score/user/:id

Leaderboard

GET /leaderboard/weekly
GET /leaderboard/monthly

Progress

POST /progress/update
GET /progress/user/:id

Analytics

POST /analytics/event

Unity and WebGL both use this API.

★ 6. UNITY IMPLEMENTATION (Short Example)

Login from Unity

```
var form = new WWWForm();
form.AddField("email", email);
form.AddField("password", password);

UnityWebRequest request = UnityWebRequest.Post("https://api.com/auth/login",
form);
yield return request.SendWebRequest();

var response = request.downloadHandler.text;
```

Sending Score from Unity

```
var scoreData = new ScorePayload {
    score = currentScore,
    level = currentLevel,
    xp = currentXP
};

string json = JsonUtility.ToJson(scoreData);
UnityWebRequest req = new UnityWebRequest("https://api.com/game/score/update",
"POST");
byte[] body = Encoding.UTF8.GetBytes(json);

req.uploadHandler = new UploadHandlerRaw(body);
req.downloadHandler = new DownloadHandlerBuffer();
req.SetRequestHeader("Content-Type", "application/json");
req.SetRequestHeader("Authorization", "Bearer " + token);

yield return req.SendWebRequest();
```

Unity and WebGL will behave the same.

★ 7. WEB-GL INTEGRATION

WebGL build will also use the same `UnityWebRequest`

But must handle CORS:

- Backend must allow WebGL domain
 - Enable HTTPS
 - Use compressed builds
 - Handle large payloads
-



8. USER DASHBOARD FEATURES

Dashboard shows:

- Profile
 - Total Score
 - Level progress
 - Badges
 - Leaderboard rank
 - Game history
 - Analytics
 - Certificates (optional)
-



9. SECURITY

- Never trust Unity client data
 - Validate on backend
 - Anti-cheat checks
 - Score verification rules
 - Rate-limiting API abuse
 - JWT expiry + refresh
 - Hash passwords, no plain text
-